Lifeguard – Second Delivery

A distributed system for low-power, non-invasive patient health monitoring and intervention

Sara G, 2063650
Annie Jain, 2252767
Syrym Khakimzhan, 2178126

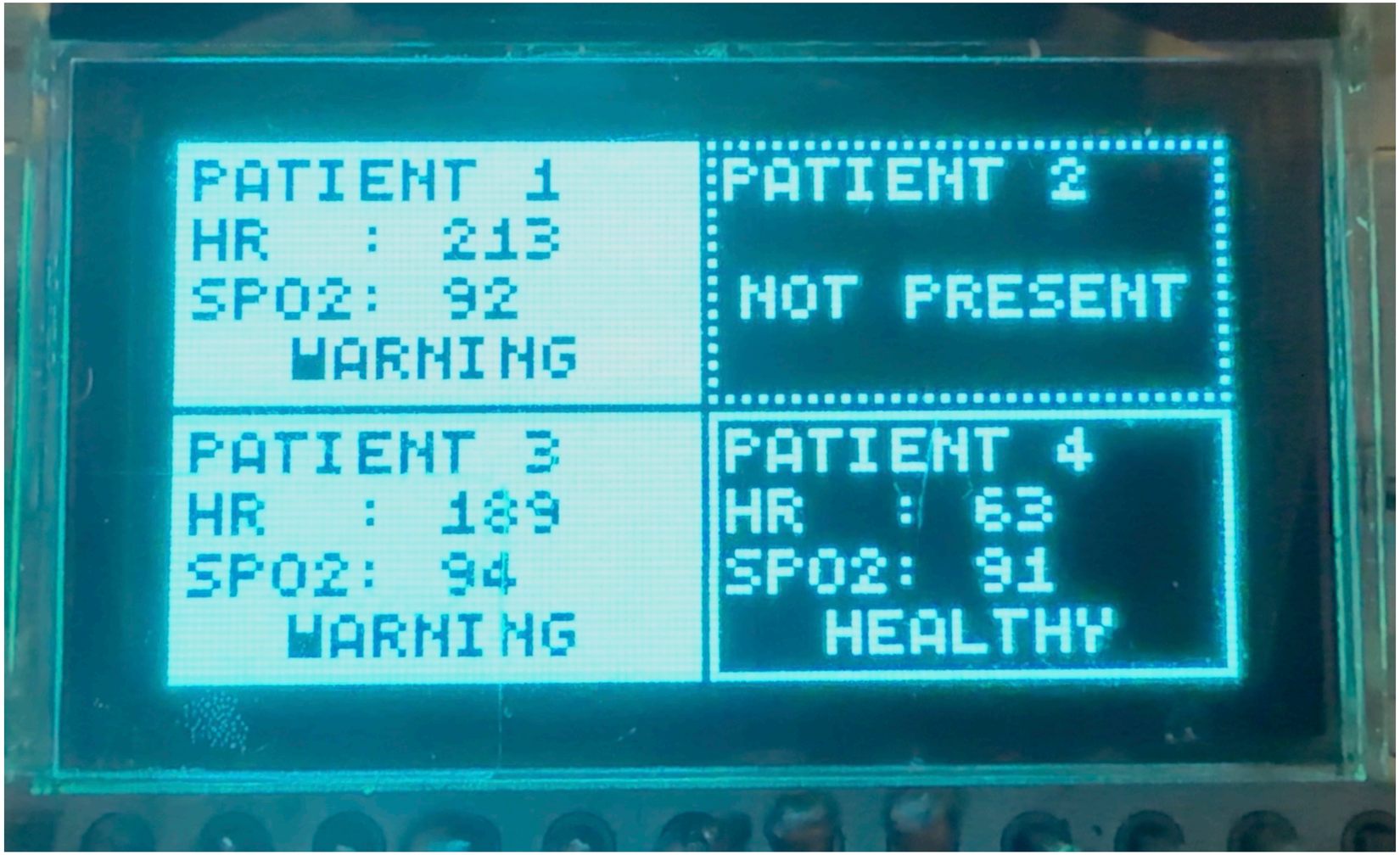
Edge Components - Hub

- alert: powers the warning/critical LED and buzzer by reading “alert state” updates from a FreeRTOS queue
- comm_hub: calls LMIC to handle incoming packets from straps, unpacks them into an internal format, filters out packets not meant for this hub (based on LoRaWAN port and our own room numbers) and sends them to an “incoming packet” queue
- monitor: handles real-time visualization of patient data: a rudimentary software 2D drawing library is provided as well as a PatientData struct and a drawing routine based on it to be used directly by the main app
  - Because of the very small 128x64 Heltec OLED screen we use for this demo, we made a custom 4x5-pixel font to pack up to 4 patients’ HR, SpO2 and warning statuses into the available space

Edge Components - Hub

- main: connects everything else and processes incoming data in real time, evaluates warning/critical conditions and feeds data into the monitor and alert system
  - Since the group hasn’t been able to acquire a LoRaWAN gateway yet, the comm_hub task is started but the queue is then fed by auxiliary code that produces random mock data

Hub in Action



Edge Components - Strap

- max30102: interfaces with the strap's MAX30102 sensor by configuring it and reading individual samples; also provides Maxim's code to process the raw IR/red samples into HR and SpO2 data
  - at the moment polling is used, the final implementation will use "queue almost full" interrupts fired around once a second
- comm_strap: reads packets from an "outgoing packet" queue, packs them into the same internal format as the hub and sends them using LMIC
- main: invokes the max30102 reading routines, handles sample aggregation and runs the HR/SpO2 algorithms as needed, then dispatches packets reporting the HR/SpO2 values to comm_strap to be sent to the hub

Strap in Action

```
I (108798) HR-SpO2: HR = 115, SpO2 =  99
I (108798) Comm: Sending packet (kind = 0): 31b9800
I (112812) HR-SpO2: HR = 107, SpO2 =  99
I (112812) Comm: Sending packet (kind = 0): 31b5800
I (116827) HR-SpO2: HR = 115, SpO2 =  99
I (116827) Comm: Sending packet (kind = 0): 31b9800
```

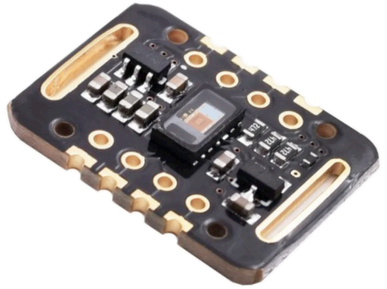
Edge Components - Common

- comm_common: provides the packet format for strap-hub communication and common LMIC code/configuration
- lmic: a LMIC port to ESP-IDF (detailed later)
- ina219: currently unused, will be later used to use one or more INA219s as i2c devices in order to measure components' and/or the ESP32's own power usage

External Code Used

- MAX30102 HR/SpO2: C code provided by Maxim (manufacturer of the sensor) for processing of the raw sensor data into HR/SpO2 values
  - It combines the photodiode responses from both the IR LED and the red LED, removes noise and identifies peaks in 4 seconds of samples at 25 samples/second
  - Chosen because it’s already calibrated for our specific sensor, however other methods might be more precise
- LMIC: MCCI-Catena’s version of LMIC was used, and its entire HAL was ported from Arduino to ESP-IDF in order to lower power usage by harnessing already-present ESP32 drivers and eliminate busy loops thanks to FreeRTOS scheduling and (more) hardware interrupts

LoRaWAN™

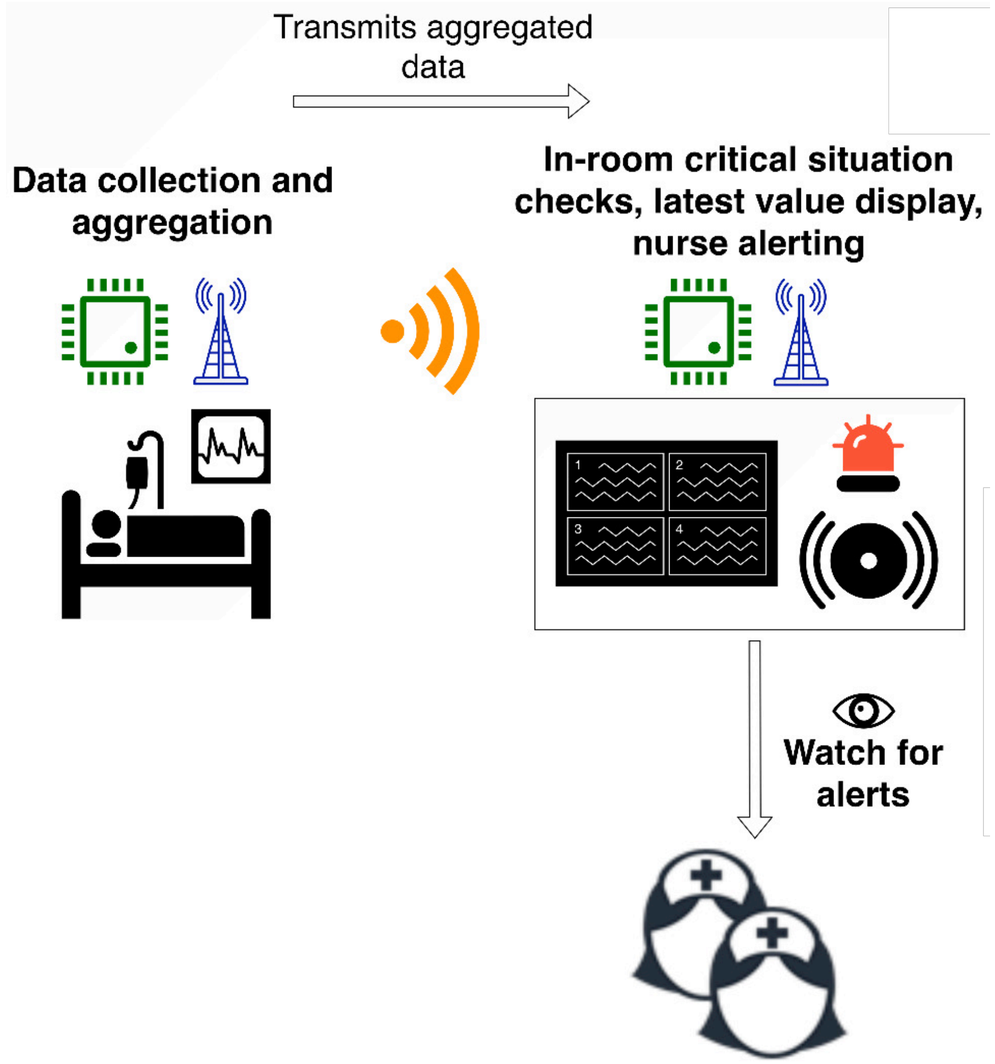


Power Draw - Strap

- MAX30102 LEDs: 20 mA while in use
- MAX30102: up to 1.2 mA while in use, up to 10 µA while shutdown
- ESP32:
  - Normal use: up to 500 mA as needed (we want to stay in this state as little as possible, removing the MAX30102 reading busy loop will help)
  - Light sleep: around 240 µA
  - Deep sleep: 7-8 µA
- LoRa TX: up to 200 mA while in use
  - Since our duty cycle is already at most 1%, this leads to at most 2 mA of average current
- ➜ we're currently using an average of 23.2 mA + the ESP32's power draw
  - Remember that we want **at most 15 mA** (50 mW at 3.3 V) so more optimizations (and empirical measurements) are needed, we can send less power to the LEDs, …

Power Draw - Hub

- ESP32: same as for the strap
- OLED screen: up to 350 mA, depending on brightness and amount of “on” pixels; right now we’re using half brightness and using around 1/5th of pixels, so we get an estimate of around 35 mA
- Buzzer and LED: up to 20 mA each, but not active in normal conditions
- LoRa RX: up to 90 mA while in use
  - Since our duty cycle is already at most 1%, this leads to at most 0.9 mA of average current
- ➜ we’re currently using an average of ~35.9 mA + the ESP32’s power draw
  - Remember that we want **at most 30 mA** (100 mW at 3.3 V), so for this too we need to look into empirical measurements and viable optimizations


Transmits aggregated data
Data collection and aggregation
In-room critical situation checks, latest value display, nurse alerting
1
2
3
4
Watch for alerts

Packet Format

- We have an “internal” packet format which uses normal C++ types and a “network” packet format which compresses data as much as possible
- 26 bits of payload when packed (in addition to the LoRaWAN header):
  - **Patient**: 2 bits
  - **Room**: 8 bits
  - **Packet kind**: 1 bit (although only “heart data” packets exist)
  - If “heart data” packet:
    - **Heart rate**: 8 bits (0-254 BPM allowed, 255 = N/A)
    - **SpO2**: 7 bits (0-100% allowed, 127 = N/A)
- Around 200 ms of airtime per transmission at SF9 ➜ ok if we lower the transmission rate to every 30 seconds

Planned Improvements

- Acquire a LoRaWAN Gateway ☺
- Use interrupts when interfacing with the MAX30102 to avoid looping and having to keep the processor awake
- Evaluate whether HR/SpO2 data has significant updates directly on the strap; if it doesn’t, aggregate it to send packets to the hub less frequently and avoid “spam”
- Harness the same processing to dynamically adjust the strap’s HR/SpO2 sample rate (for example, only sample every 30 seconds if it’s been stable for a long time) in order to optimize battery life

Planned Improvements

- Attempt to run an embedded machine learning model on the hub in order to notice positive or negative trends in patients' health data and issue warnings or critical alerts

Other Possible Improvements

- Adopt ESP32 deep sleep modes to reduce power usage (FreeRTOS can only do light sleep)
  - We can probably do this on the strap because its only “timed” source is the HR/SpO2 sensor, which can emit interrupts as needed
- Follow through on a basic central dashboard accessed from a “normal” computer
- On the hub, use on-device, per-patient reinforcement learning in order to personalize the warning prediction model to a patient’s specific physiology
  - Start from the common prediction model as a default and train it further over time
  - If implemented, this will require external feedback mechanisms to signal the ground truth about the patients’ health to the model